TierForge

Resilient Bulk Store Scoring & Tiering - System Design Document

Take-Home Exercise - Tech Lead Hiring

1. Overview

TierForge ingests a large CSV of stores, enriches each store with business metrics (estimated monthly footfall, estimated monthly revenue, store size in sqft) via a slow, rate-limited, and occasionally unreliable external Enrichment API, then scores and tiers every store (Large / Medium / Small) based on user-configurable weights and thresholds.

The central design goal is a clean separation between the slow, unreliable enrichment step and the fast, deterministic scoring step. Enrichment is an asynchronous, resumable job engine backed by PostgreSQL; scoring is a pure function over data already stored in the database and never calls the Enrichment API.

2. Tech Stack

Layer	Technology	Rationale
Frontend	React 18 + TypeScript + Vite + TanStack Query	Type-safe UI; TanStack Query gives polling, caching and invalidation for job progress with minimal boilerplate.
Backend	Node.js + Express + TypeScript	Fits the preferred stack; Express keeps the HTTP layer thin so the job engine stays framework-agnostic and testable.
Database	PostgreSQL	Single source of truth for all job state; SELECT ... FOR UPDATE SKIP LOCKED provides a correct, crash-safe work queue without extra infrastructure.
Testing	Vitest (41 unit tests)	Fast unit tests around retry/backoff, the scoring formula, and tiering boundaries.
Enrichment API	Provided simulator (run locally)	Treated as an uncontrolled third-party dependency: rate limit 5 req/s, ~10% transient 500s, ~2% 45–60s delayed responses.

3. System Architecture

The system has three runtime pieces: the React frontend, the Express backend, and PostgreSQL. The Enrichment API simulator runs alongside as an external dependency.

```
Browser (5173)  -->  Express API (4000)  -->  Enrichment API (8000)
                              |
                              v
                         PostgreSQL
```

3.1 Key architectural decision: enrichment vs. scoring

Enrichment and scoring are strictly separated. Enrichment writes raw metrics into the enrichment_results table and nothing else. Scoring reads only that table, applies the user's bars/weights/tier thresholds as a pure function, and writes results into store_scores. Consequences:

- Scoring is instant and re-runnable: changing bars, weights, or tier thresholds re-computes everything from stored data without a single API call.
- The flaky dependency is quarantined behind one module; the rest of the system never sees its failure modes.
- Each side is independently testable: scoring tests need no HTTP mocking.

3.2 Data model

Table	Purpose
jobs	One row per enrichment job: status (pending/running/completed/failed), counters, terminal failure reason.
job_stores	One row per store per job: the unit of work. Carries a lease token + lease expiry so a worker can claim it exclusively and a silent worker's claim can be reclaimed.
enrichment_results	Raw metrics per store, written once (idempotent upsert). The only input scoring ever reads.
enrichment_attempts	Audit log of every API attempt: attempt number, outcome, latency, error. Powers per-store failure reasons in the UI.
scoring_runs	Each scoring submission: the config snapshot (bars, weights, tier thresholds) so results are reproducible and auditable.
store_scores	Computed score + tier per store per scoring run.

4. Backend Design

4.1 Job engine

The job engine is a set of in-process workers that claim work from PostgreSQL rather than an in-memory queue. Claiming uses SELECT ... FOR UPDATE SKIP LOCKED: a worker atomically takes the next pending store, stamps it with a lease token and expiry, and processes it. This gives three properties directly from the database:

- No lost work: a row is only marked done after the result transaction commits.
- No duplicated work: two workers can never claim the same row; SKIP LOCKED makes concurrent claims safe by construction.
- Crash safety: all state lives in Postgres, so a killed process leaves no in-memory state behind.

4.2 Leases and recovery from silent workers

Every claimed store carries a lease with an expiry. If a worker hangs or dies mid-call, a reaper (running every 2 seconds) finds rows whose lease has expired and returns them to pending so another worker retries them. On process boot, resumeRunningJobs() re-attaches to any jobs left in running state, so a hard restart (verified with kill -9 mid-job) continues without redoing completed work or losing in-flight work.

4.3 Handling the unstable Enrichment API

The simulator's documented failure modes map to specific mechanisms:

Failure mode	Mechanism
Global 5 req/s rate limit (429)	One shared, process-wide paced rate limiter set to 4.5 req/s - deliberately below the limit, so the system never relies on 429s for pacing. A 429 that still occurs refunds the attempt budget and is retried.
~10% transient 500s	Capped exponential backoff with jitter. Retries are bounded; the cap prevents infinite loops.
~2% of calls take 45–60s	Soft timeout (8s): the worker stops waiting, releases the store, and moves on. Hard timeout (75s): the underlying request is aborted so a hung call can never block a worker forever.
Late answer from a 'dead' attempt	Committed idempotently: the result is written with a primary-key + ON CONFLICT DO NOTHING upsert, so whichever attempt lands first wins and the late one is a harmless no-op. Nothing is ever double-counted.
Systemically broken run	Circuit breaker over a sliding window of the last 60 outcomes: if 90% are failures, the job stops retrying and enters a clear terminal failed state with a human-readable reason, instead of looping forever.
4xx (permanent) errors	Fail the store immediately - retrying a deterministic bad request wastes budget.

4.4 Idempotency and correctness

Every write path is safe to repeat: result commits are upserts keyed on (job, store), attempt logging is append-only, and job counters are derived from row state rather than incremented blindly. This is what makes late responses, retries, and restarts safe to combine.

4.5 Scoring service

Scoring is a pure function: score = sum of weights for every metric bar the store clears; tier = Large if score >= large threshold, else Medium if >= medium threshold, else Small. Config is validated before running (weights must sum to 100%, large threshold must exceed medium). The worked example from the brief (stores A/B/C scoring 100%/50%/20%) is covered by a unit test.

5. Frontend Design

5.1 Stack and structure

React 18 + TypeScript + Vite, with TanStack Query for server state. The UI is a single dashboard composed of four panels: Upload, Job Progress, Scoring Config, and Store Table. A dark 'forge' visual theme gives the tool a distinct identity.

5.2 Step 1 - Upload

CSV upload with client-side header validation before the file is accepted (required columns: store_id, store_name, address, city, state, country). Malformed files are rejected with clear, detailed errors rather than failing silently server-side.

5.3 Step 2 - Job progress

Once a job is submitted, the Job panel polls the backend and shows live counts: enriched / failed / pending. For every store that ultimately failed, the failure reason (from the attempt audit log) is visible, satisfying the requirement to explain failures, not just count them.

5.4 Step 3 - Scoring configuration

The user sets a threshold bar and a weight per metric, plus the Large/Medium tier cutoffs. The form validates live: weights must sum to 100% and the Large threshold must exceed the Medium one; invalid configs cannot be submitted. Submitting re-runs scoring against stored data only - the UI makes clear this is instant and never re-calls the Enrichment API, so users can freely experiment with the tier breakdown.

5.5 Step 4 - Dashboard

The dashboard shows the tier breakdown (counts of Large / Medium / Small) and a paginated store table with name, the three metrics, computed score, and tier, filterable by tier. Pagination keeps the 5,000-store dataset responsive.

5.6 Live updates: polling, not push

A 400-store job takes a couple of minutes, and the person who started it is watching the numbers climb. Rather than hold a connection open, the UI simply asks the backend again every 1.5 seconds while a job is running, and stops asking the moment the job reaches a finished state. With TanStack Query that comes down to one line: refetchInterval returns 1500 while the status is “running” and false otherwise. The same hook that loads the job when the page opens is the one that keeps it fresh, so there is no second mechanism to keep in sync, and no timer still ticking after the work is done.

Why poll instead of pushing over a socket? Because the numbers on screen already live in Postgres. Workers update the job’s counts as they go, so a refresh is a cheap read of a table we are querying anyway, and the answer is the current truth rather than whatever the last event happened to say. A push channel would have to be built, kept alive through proxies, and reconciled against the database anyway. For one job at a time, a request every 1.5 seconds costs almost nothing and keeps the frontend small. If this grew to many concurrent jobs, server-sent events would be the better shape - a swap behind one query hook, not a rewrite.

6. API Reference

All endpoints live under the /api prefix and speak JSON (the upload endpoint is multipart). CORS is enabled so the Vite dev server can call the API directly, and a single central error handler normalises every failure into a consistent { error, details? } body: CSV or configuration validation problems return 422 with per-row messages, malformed requests return 400, oversized uploads return 413, and unexpected failures are logged server-side and return 500 without leaking internals.

The nine endpoints below are the complete API surface. Three of them (create job, latest job, job detail) return the identical job-view shape - status, progress counts (pending / in_flight / succeeded / failed), and attempt statistics - so the frontend shares one type and one polling loop across all three.

Endpoint	What it does
GET /api/health	Liveness probe returning {"status":"ok"}; used by the Docker health check and the frontend's connectivity banner.
POST /api/jobs	Uploads a store list as a CSV file in the multipart "file" field. Validates the required columns and the row cap (max 5,000 stores), creates the job with every row seeded as pending, and starts the enrichment engine. Returns 201 with the job view; 422 with per-row validation errors for a bad CSV; 409 if another job is still active.
GET /api/jobs/latest	Returns the most recent job with progress counts and attempt statistics, or null if no job exists yet. The frontend calls it on load to restore its state after a refresh or a server restart.
GET /api/jobs/:id	Full view of one job: status, failure reason, timestamps, progress counts (pending / in_flight / succeeded / failed) and attempt outcome statistics from the audit log. 404 for an unknown id. Polled while a run is in progress.
POST /api/jobs/:id/cancel	Cancels a running job: no new work is claimed and in-flight leases are handed back to pending so late responses can still be committed idempotently. Returns the updated job view; 409 if the job has already finished.
GET /api/jobs/:id/failures	Paginated list of the failed stores: store id and name, attempt count, number of rate-limit (429) hits, and the last error message. limit is capped at 500; offset-based paging. Powers the failure detail panel.
POST /api/jobs/:id/scoring	Runs scoring for the job. The body carries the configuration: the three metric weights, which must sum to 100, and the tier thresholds, where Large must sit above Medium. The computation is pure - it reads only the stored enrichment results, never the external API - and atomically replaces the previous tiering inside one transaction. Returns the run summary; 422 with validation messages for an invalid configuration.
GET /api/jobs/:id/scoring	Returns the latest scoring run: run id, the configuration used, how many stores were scored, and the Large / Medium / Small breakdown. Returns null if scoring has not been run yet.

Endpoint	What it does
GET /api/jobs/:id/stores	Paginated store table with the three enriched metrics, the computed score, tier, per-store status and last error. Supports a tier filter, case-insensitive text search across name / id / city, and page plus pageSize (max 200). Sorted by score descending so the highest-value stores come first.

Notably, the API is deliberately small: enrichment progress is a read-only concern (the job endpoints), and scoring is a separate command endpoint - nothing in the API mixes the two, mirroring the separation enforced inside the engine.

7. Resilience Verification

Each failure mode was verified end-to-end against the real simulator:

- 400-store job: 400/400 enriched, zero 429s hit (paced limiter), late answers accepted exactly once.
- Simulator down: job failed fast with a clear circuit-breaker reason instead of retrying forever.
- kill -9 mid-job: after restart the job resumed and finished 150/150 with no duplicates.
- Scoring matches the brief's worked example; malformed CSVs rejected with detailed errors.
- 41 unit tests pass via a single documented command (npm test), covering backoff, the scoring formula, and tier boundaries.

8. Known Limitations

- Single-node deployment: the lease model already supports multiple worker processes, but only one was run.
- No authentication — out of scope for the exercise.
- The bonus items (concurrent jobs under one shared rate limit, full restart resumability) were both implemented; the shared limiter is process-wide, so multi-process deployments would need a distributed limiter.